Evolutionary Adversarial Reprogramming of Growing Neural Cellular Automata

Aaron Bussche David Magaram Jason Kasdiran

Group 28

June 13, 2026

Introduction

Neural Cellular Automata (NCAs) are small, shared-weight networks that grow images from a single seed pixel and can regenerate after damage, making them a promising model for robust, self-organizing systems. Their perceived robustness has led to NCAs being deployed as a defensive component in larger architectures: Xu et al. [2] use NCA layers as plug-and-play adaptors inside Vision Transformers to make them more resistant against adversarial samples. Whether NCAs themselves can be subverted is therefore not just of theoretical interest but relevant to the security of systems that rely on them.

Randazzo et al. [3] and Cavuoti et al. [4] demonstrated targeted attacks that, for example, can make a generated lizard image lose its tail or change color. However, every published attack on NCAs relies on white-box, gradient-based methods that require full access to the model's weights and backpropagation through the rollout. Robustness claims are so far tested only against this strong threat model. Additionally, focusing on the nature-inspired roots of NCAs, a white-box is not what a realistic adversary, for instance in a biological or bioengineering setting, would plausibly have access to.

Problem Statement

In this project we investigate whether a query-only, black-box attacker using evolutionary search can reproduce the same targeted reprogramming of a Growing NCA by changing it towards the target image, and how close does it get to the gradient-based ceiling.

The goal is to find a 16×16 state-perturbation matrix that when applied to every cell's 16-dimensional hidden state at every timestep forces the NCA to grow into the target image rather than to its original form (blue lizard instead of a green lizard), which is the same attack surface used by Randazzo et al. [3]. In our approach the attacker can only observe the final RGB output and optimize the L2 distance to a chosen target image using the CMA-ES evolutionary algorithm, without ever accessing model weights or gradient information.

We evaluate if our black-box attack can influence the Growing NCA in a similar way as the white-box approach by comparing the approaches by their final grown-image loss and visual output quality. If the evolutionary attacker approaches the gradient ceiling, it suggests that NCAs may be vulnerable and less robust than previously assumed. However, if it fails to closely approach that ceiling, then the NCAs may be considered safe from more realistic attack types and safe to be used in public-facing, but not open-weights, machine learning systems.

Related Work

Mordvintsev et al. [1] introduced the Growing NCA model and showed that, through a sample pool training strategy and exposure to damage during training, the learned local update rules become robust enough to regenerate damaged parts. Due to this robustness, NCAs have also been used defensively: Xu et al. [2] inserted NCA layers between vision transformer blocks to serve as an armor against adversarial attacks.

On the offensive side, Randazzo et al. [3] investigated white-box vulnerabilities of Growing NCAs and showed that, while they are more resistant to local injections, they can still be reprogrammed using global state perturbations applied to every cell at every step. Similarly, Cavuoti et al. [4] demonstrated that injecting a small number of adversarial cells into the grid can drastically alter the grown image. Both studies rely on gradient-based optimization with full access to model weights.

Covariance Matrix Adaptation Evolution Strategy (CMA-ES), introduced by Hansen [5], is an algorithm for non-linear, non-convex black-box optimization that maintains a population of candidates and iteratively shifts them toward high-fitness regions. While evolutionary black-box attacks are commonly used to evaluate traditional image classifiers, they have not yet been applied as an adversarial tool against cellular automata.

Methodology and Approach

Motivation and description of your approach, including: methods, algorithms, evaluation/testing procedures, experimental setup, etc. Ensure that the work is reproducible given this description alone.

To assess the true vulnerability of NCAs under a realistic threat model, we constrain the attacker to a strict black-box setting. The attacker cannot access the internal weights of the pretrained NCA, nor can they perform automatic differentiation through the grid's temporal evolution. Instead, the attacker can only provide an initial state (or intervening perturbation) and observe the final converged visual output. Because the fitness landscape of an NCA rolled out over hundreds of steps is highly non-convex and non-differentiable from a black-box perspective, we utilize CMA-ES. This evolutionary strategy is well-suited for our setup because it is derivative-free, robust to noisy objective functions, and scales well to our specific parameter space.

Experimental Setup

We target a pretrained Growing NCA model optimized to generate a specific 2D image from a single seed pixel (e.g., the "lizard" emoji model) introduced by Mordvintsev et al. [1]. The NCA operates on a W × H (40 × 40, padding 32) grid of cells, where each cell (pixel) has a continuous 16-dimensional state vector $x \in \mathbb{R}^{16}$. Among the 16 channels, 3 encode RGB color, 1 encodes transparency (Alpha), and the remaining 12 are hidden.

The gradient-based attack from Randazzo et al. [3], which optimizes M directly via backpropagation through the full NCA rollout using MSE loss and the Adam optimizer, with complete access to model weights, serves as the performance ceiling against which our black-box approach is measured. Following this framework, we restrict our attack to a global, linear state perturbation. At each timestep of the simulation, before the NCA's convolutional update rules are applied, every cell's state vector is multiplied by a global transformation matrix $M \in \mathbb{R}^{16\times 16}$. To maintain stability in the cellular dynamics, we restrict M to be a symmetric matrix. A symmetric 16×16 matrix contains exactly 136 unique parameters (the upper triangular elements, including the diagonal). This 136-dimensional vector space forms the search space for our CMA-ES algorithm.

To evaluate the impact of the starting distribution on the evolutionary search, we initialize the population center from three distinct points:

- **The Identity Matrix:** This represents a "zero-damage" starting point, allowing the evolutionary search to incrementally discover minimal perturbations.
- **A Random Matrix:** Initialized via a standard normal distribution to test the algorithm's ability to converge from an arbitrary, highly destructive state.
- **The Baseline Gradient Matrix:** The final output matrix derived from the white-box gradient descent method used by Randazzo et al., establishing a benchmark for convergence speed and local minimum trapping.

For the initial standard deviation (σ_0), which dictates the initial search volume, we conduct a sweep across multiple magnitudes (e.g., $10^{-2}, 10^{-3}, 10^{-4}$). A smaller σ_0 is critical when starting from the identity matrix to prevent immediate catastrophic failure of the NCA rules.

During CMA-ES training, the NCA is rolled out for a randomly sampled number of steps in $[64, 96]$ per generation. For evaluation, a fixed 96 steps are used. At each generation, the CMA-ES produced population of 64 candidate vectors. Each candidate is evaluated by rolling out the NCA from a batch of `batch_size`$= 4$ states taken from a sample pool of 1024 states and averaging the resulting loss. We apply step-size decay to avoid making disruptive jumps during search. There, σ is multiplied by `decay_factor` $= 0.9$ every `decay_freq` $= 175$ generations. The chosen hyperparameters were selected first by manual experiments, but later by 2 narrowing Optuna sweeps depicted in Figure 1 and Figure 4.

Figure 1 displays that the run clearly favored a batch size of 4 and population size of 32 or 64. It can also be said that larger moderate decay frequency was more favorable than lower one. Figure 2 shows the hyperparameter importance for the first Optuna sweep. It reveals that batch_size is the most important hyperparameter by a significant amount. In the second sweep in Figure 3, the winners from the previous Optuna sweep were re-introduced, such as batch_size 4, so we could find the remaining best hyperparameters. This run favored population size of 64, a higher sigma0, and a higher decay frequency.

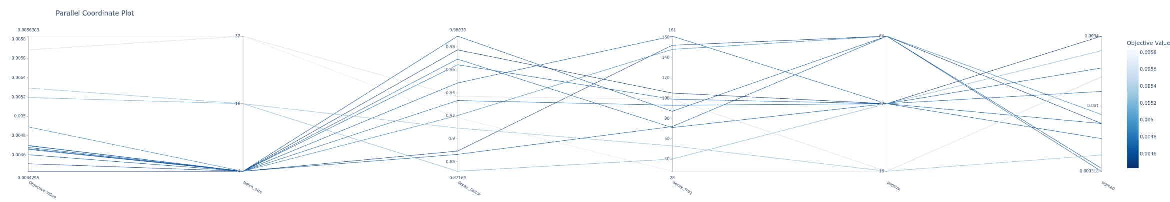

Figure 1: First optuna sweep. It ran 30 times in 20 minutes with early pruning after 5 minutes.

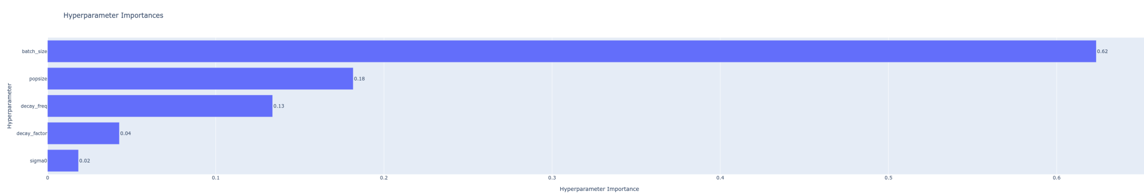

Figure 2: Hyperparameter importance during the first Optuna sweep.

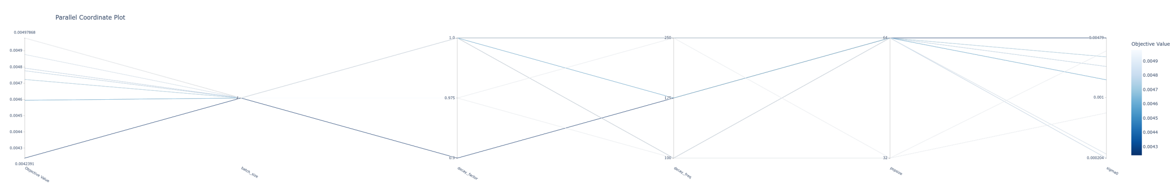

Figure 3: Second optuna sweep with less options as the winners from the first round were re-introduced. It ran 20 times in 20 minutes with early pruning after 5 minutes.

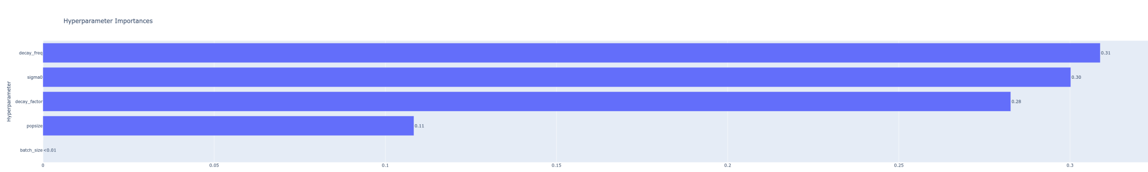

Figure 4: Hyperparameter importance during the second Optuna sweep.

Based on the two Optuna sweeps, our final algorithm uses the parameters displayed in table 1.

Parameter	Value
sigma0	0.0048
batch_size	4
popsize	64
decay_factor	0.9
decay_freq	175

Table 1: Hyperparameter values used for CMA-ES optimization.

Evaluation Metrics

While the Mean Squared Error (MSE, or L_2 distance) is utilized as the primary fitness function during the CMA-ES optimization process due to its computational efficiency, it is widely recognized that pixel-wise distances correlate poorly with human visual perception. A perturbation matrix may achieve a low MSE while simultaneously producing an image with unnatural artifacts or structurally disjointed features. Therefore, to rigorously evaluate and compare the outputs of the black-box evolutionary attack against the baseline gradient-based attack, we employ a suite of quantitative metrics focusing on structural, perceptual, and frequency-domain similarities.

Structural Similarity Index Measure (SSIM): Introduced by Wang et al. [7], SSIM evaluates the visual impact of luminance, contrast, and spatial structural dependencies between two images. Unlike MSE, which estimates absolute errors, SSIM considers image degradation as perceived change in structural information. We calculate the SSIM over the RGB channels with a maximum pixel value of 1.0. A higher SSIM value (closer to 1.0) indicates better structural fidelity to the target image.

Learned Perceptual Image Patch Similarity (LPIPS): To quantify semantic and perceptual similarity, we utilize the LPIPS metric [8]. By passing both the target image and the generated NCA output through a pretrained deep convolutional network (in our case, AlexNet) and comparing the internal feature activations, LPIPS effectively measures whether the generated image "looks" semantically identical to the target. Images are normalized to the $[-1, 1]$ range prior to extraction. A lower LPIPS score indicates that the adversarial image is perceptually closer to the intended target.

Target-Specific Semantic Score (TSS): While automated metrics like SSIM and LPIPS capture structural and deep feature similarities, they can occasionally overlook distinct morphological traits critical to the specific target image (e.g., the accurate formation of individual limbs or highly localized color patterns). To bridge the gap between qualitative human perception and quantitative evaluation, we implement a Target-Specific Semantic Score. This metric systematically assesses five key features of the generated morphology: Granularity, Bodily Parts, Blueness, Dorsal Stripe, and Head Features. Each feature is evaluated on a discrete scale from 1 (highly degraded) to 4 (excellent target match). The final composite score $\sum TSS$ is then min-max normalized to yield a continuous value $TSS_{norm} \in [0, 1]$ according to the following formula:

$$TSS_{norm} = \frac{\sum TSS - 5}{15}$$

where a score closer to 1.0 indicates superior structural and morphological alignment with the target. The comprehensive evaluation rubric detailing the specific criteria for each score is provided in table 5.

2D Fast Fourier Transform (2D-FFT): Adversarial reprogramming of cellular automata relies on altering local convolutional updates, which frequently introduces high-frequency structural noise (e.g., checkerboard artifacts) not heavily penalized by spatial loss functions. To quantify these artifacts, we perform a 2D Fast Fourier Transform (2D-FFT) on the grayscale representations of the generated and target images. We compute the mean absolute difference of their log-magnitude spectra:

$$D_F = \frac{1}{H \times W} \sum_{u,v} \Big| \log(1 + |\mathcal{F}(\text{img}_{\text{target}})_{u,v}|) - \log(1 + |\mathcal{F}(\text{img}_{\text{grown}})_{u,v}|) \Big|$$

where $\mathcal{F}$ denotes the discrete 2D-FFT.

Furthermore, because our targets involve localized morphological changes (such as the removal of a lizard's tail or head), a global Fourier transform may average out highly localized frequency discrepancies. To address this, we implement a partitioned spectral difference metric: the images are divided into a 2×2 grid (four quadrants), and the spectral difference is calculated independently for each block and then averaged. This allows us to precisely capture localized artifacting around the explicitly altered regions of the target shapes.

Different Fitness Functions

At each generation, every candidate perturbation matrix is evaluated by rolling out the NCA for a batch of states and computing a scalar distance between the generated output and the target image. CMA-ES then uses this to update its search distribution.

Five different fitness functions were evaluated in total. All were evaluated on the same color change task across 500 generations using identical parameters found in table 1. Fitness functions were selected based on their relevance to this task, with MSE serving as the baseline, and evaluated using the evaluation metrics mentioned in earlier.

Mean Squared Error (MSE): MSE measures the average of the squared differences between the actual and predicted values. It is a pixel-wise fitness function that is applied to all 4 channels, where x is the generated image and t is the target image. It squares the errors, giving more weight to larger errors, and is used as a baseline in this project.

$$\mathcal{L}_{\mathrm{MSE}}(x) = \frac{1}{HWC} \sum_{h,w,c} \left(x_{hwc} - t_{hwc}\right)^2$$

Huber: Huber loss [9] is a robust fitness function which combines MSE and MAE. It uses MSE for small errors and switches to MAE for larger errors when δ is crossed. We use a small $\delta = 0.1$ threshold to use MSE for pixel errors smaller than 10% of the full range and MAE for larger errors.

$$\ell_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \le \delta \\ \delta\big(|r| - \frac{1}{2}\delta\big) & |r| > \delta \end{cases}$$

Weighted RGBA: This is a variant of the MSE function that gives a higher weight to the specified channels: $\mathcal{L} = \frac{1}{HWC}\sum_{h,w,c} w_c(x_{hwc} - t_{hwc})^2$. Weights were manually set to $w = 2$ for RGB and $w = 1$ for A.

Foreground-masked MSE: This fitness function is a spatially weighted variant of MSE that scales each pixel's contribution by its position in the image. It works similarly to weighted RGBA, however, all channels are weighted equally, pixels inside the body contribute fully, but other pixels contribute less, with background pixels contributing only 20%.

$$\mathcal{L}_{\mathrm{fg}}(x) = \frac{1}{HWC} \sum_{h,w,c} \big(0.2{+}0.8\, t_{hw,\alpha}\big) \cdot \big(x_{hwc}{-}t_{hwc}\big)^2$$

SSIM: As detailed in the Evaluation Metrics section, we also explore SSIM [7] as an alternative fitness function to explicitly optimize for structural and perceptual similarity.

$$\mathcal{L}_{\mathrm{SSIM}}(x) = 1 - \frac{1}{|W|} \sum_{w \in W} \frac{(2\mu_x\mu_t + c_1)(2\sigma_{xt} + c_2)}{(\mu_x^2 + \mu_t^2 + c_1)(\sigma_x^2 + \sigma_t^2 + c_2)}$$

Results

While the gradient-based model reaches better results in the same time, the loss of the NCA-ES trained adversarial attack is not too far off (figure 5). It is interesting to note that the gradient-based attack has a higher standard deviation than the evolved one.

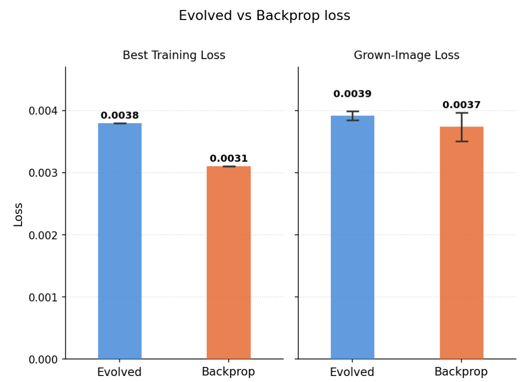

Figure 5: The best loss achieved during training and the averaged loss of the final model after training both of our implementation and the gradient-based implementation after 30 minutes of training time.

To compare the optimization efficiency of the black-box CMA-ES attack against the gradient-based baseline, we tracked the convergence of the $\log_{10}$ objective loss across three different independent variables, namely wall-clock time, algorithmic iterations, and cumulative NCA state-steps. Given the fundamentally different mechanics of population-based search versus gradient descent, relying on a single metric does not provide the full picture. Wall-clock time serves as a practical benchmark for real-world adversarial feasibility; by this measure, the evolutionary attack is highly competitive. Figure 6 shows that the gradient-based baseline completes significantly more update steps and converges to a marginally lower final loss than the CMA-ES attack. While the loss per iteration (figure 7) appears superficially comparable, this metric masks the differing computational payloads. A single CMA-ES iteration evaluates an entire population of candidates, whereas the gradient-based iteration evaluates a single mini-batch. The loss per cumulative NCA state-steps (figure 8) isolates forward-pass sample efficiency, highlighting that the CMA-ES attack requires roughly an order of magnitude more simulation steps to achieve convergence compared to the gradient-based baseline. Thus, when analyzing the loss per iteration or per cumulative NCA state-step, the inherent sample inefficiency of the CMA-ES attack becomes apparent.

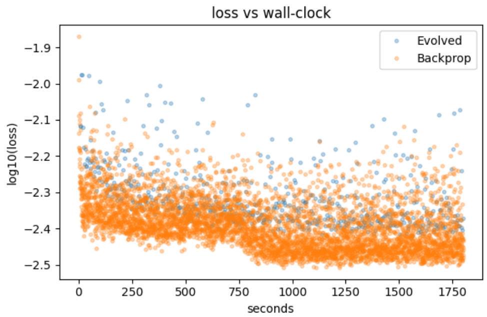

Figure 6: Optimization trajectory measured by wall-clock time (approximately 30 minutes).

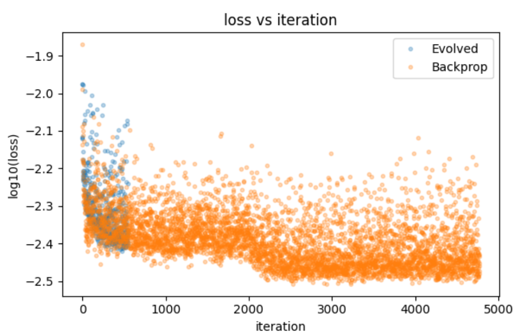

Figure 7: Optimization trajectory measured by training iterations.

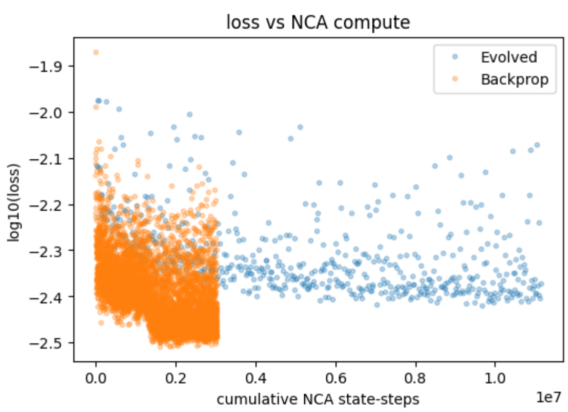

Figure 8: Optimization efficiency measured by cumulative NCA state-steps.

To contextualize the computational cost and environmental footprint of different methods, we tracked carbon emissions and energy consumption during the optimization process. For a fair comparison of convergence efficiency, each method was run until it reached a target log10 loss of -2.5.

As detailed in Table 2 and illustrated in Figure 9, the gradient approach is substantially more efficient. It successfully reached the target loss in roughly half the time and consumed approximately half the overall energy compared to the evolutionary approach.

Method	Emissions	Duration	Energy
CMA-ES	0.015	1994.738	0.043
Gradient	0.008	1024.074	0.022

Table 2: Environmental impact and computational cost of the evolutionary strategy and the gradient baseline. The table presents total carbon emissions in $kgCO_2e$, optimization duration in seconds, and overall energy consumed in kWh.

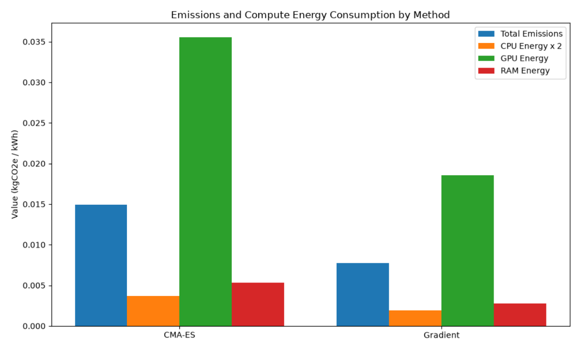

Figure 9: Comparison of total carbon emissions and energy consumption breakdown (CPU, GPU, and RAM) across the evaluated methods.

Figure 10: Left: The target image for our attack. Middle: Lizard grown from pixel with CMA-ES attack (ours). Right: Lizard grown from pixel with gradient-based attack.

Figure 11: Left: fully grown lizard, original target image. Middle: CMA-ES attack on fully grown lizard. Right: gradient-based attack on fully grown lizard

Figure 12: Gradient-based attacked Lizard during growing steps 0, 20, 40, 60, 80, 100, 120

Figure 13: Evolutionary attacked Lizard during growing steps 0, 20, 40, 60, 80, 100, 120

Visually, both the gradient-based attack and our lizard trained with CMA-ES produce noisier results than the trained NCA and the target image, which becomes obvious when looking at them closely, but not when looking at them from afar. In detail, our lizard has sharper, ragged edges and colours like the original green but also a darker blue shining through more. The gradient-attacked lizard has greater colour accuracy and superiour details (such as the position of the eyes & the white line on the back), but seems a bit blurrier. That said, it is not as easy as to say that the gradient-based attack produces a blurred version of the target image, as a blurred version of the target image is of brighter colour, with cleaner fine details (such as the eyes and tail).

Figure 14: For reference: the target image transformed down and then up 4x with linear, none, and cubic interpolation algorithms in GIMP.

Butterfly

In order to explore the capabilities of the NCA model and the CMA-ES framework, we tried to change the original lizard into a butterfly. For this, we only needed to change the target image.

Figure 15 displays 5 images from the visualization of the training run at 5 different steps, along with the target image for comparison.

Step 120 was selected because the image had the most butterfly-like shape. The overall color distribution also approximates the the target image distribution, however, the details and patterns are not present. At 500 steps, the butterfly shape is somewhat lost. After 500 steps, perturbation is turned off and the NCA starts regenerating. Because of the strange shape and pixel distribution, 2 lizards are regenerated. After 1000 steps, perturbation is turned back on. Again, after 120 steps the shape is the most butterfly-like, however, this time there are 2 lizards so we end up with 2 overlapping butterflies. At the end at 2500 steps the NCA again loses its shape in a similar fashion as at 500 steps.

Figure 15: Images from the visualization of the training run at steps 120, 500, 1000, 1120, and 2500, with perturbation off from 500 to 1000, trained for 600 steps. Target image included.

While the target image was not reached, it can be concluded that the task succeeded to some degree as CMA-ES managed to approximate the butterfly, both its shape and its orange color. This also demonstrates the CMA-ES black-box capabilities and the NCA's impressive regenerative powers.

Experiment - How long it takes to recover from an attack: Perturbation On/Off

Following the same steps as X did in their experiments, we also explored how long it takes to recover from an attack by setting perturbation on and off.

Different starter matrices

Instead of starting from identity as attack matrix, we also tried starting from white noise & the 8000 step gradient-optimized matrix. We used the same hyperparameter values we found using optuna to train a 500-generation attack starting from the identify matrix.

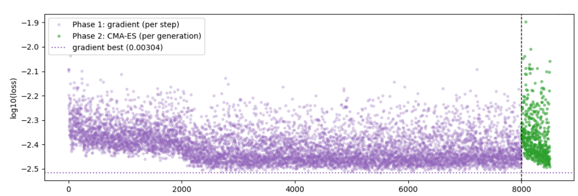

Figure 16: CMA-ES does not visibly improve upon the gradient-baseline training loss within 500 generations. There is a significant jump upwards in loss after the switch, likely caused by sigma0 being too high and too big changes being made.

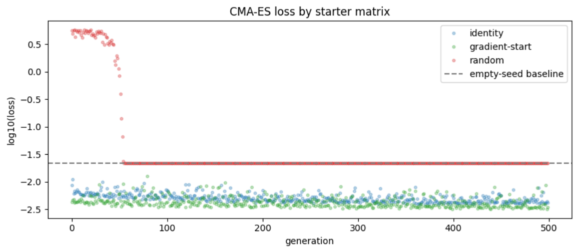

Figure 17: While the model trained starting from the identity matrix and the model trained from the gradient-result-matrix continued improving, the white-noise matrix start only managed to not be worse than producing a white canvas and got stuck there. The baseline here is the loss for the starter image of white canvas with a single black pixel in the middle.

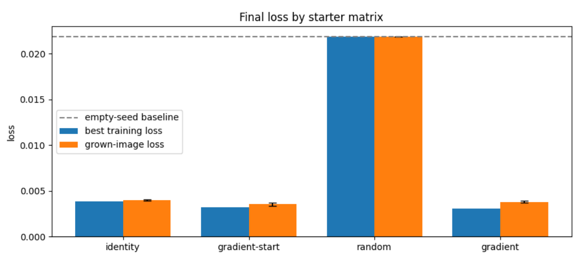

Figure 18: In averaged loss after growing a full lizard from a pixel, the random matrix start sits at the empty baseline. Identity comes third place at 0.0040, gradient-only is second at 0.0038 (+- 0.000086) and the best result comes from further training the gradient output with CMA-ES at 0.0036 (+- 0.000177).

Figure 19: Target (nr.1) and gradient-trained (nr. 4) lizards among CMA-ES trained ones. The third one started training on the gradient-trained matrix. The output from the white-noise trained matrix is just white, while the other ones clearly show a lizard. The Lizard trained from the gradient-trained baseline matrix seems to have the best colour accuracy, while the gradient-trained one has the most detailed white spine.

Method	SSIM ↑	LPIPS ↓	2D-FFT ↓	TSS ↓
Identity	0.932	0.081	0.437	0.4
Gradient-start	0.941	0.070	0.435	0.5
Gradient	0.934	0.074	0.439	0.6
White Noise	0.857	0.551	1.488	0.0

Table 3: Quantitative comparison of evaluation metrics across different attack initializations and the gradient baseline. Higher SSIM, lower LPIPS, lower 2D-FFT, and lower TSS indicate better visual fidelity to the target image.

Different Fitness Functions

Table 4 displays the evaluation metrics for the explored fitness functions.

Fitness Functions	SSIM ↑	LPIPS ↓	2D-FFT ↓
MSE	0.925	0.096	0.541
Huber	0.935	0.097	0.495
Weighted RGBA	0.934	0.103	0.544
Foreground-masked MSE	0.927	0.117	0.576
SSIM ⋆	0.950	0.197	0.362

Table 4: Fitness functions with their evaluation metrics. (lower) or (higher) determine if a smaller or larger scores are better. ⋆ means that the lizard did not turn blue.

As can be seen from the table, all losses perform similarly well. However, out of all the losses, Huber performs the best overall, achieving the highest SSIM of 0.9349, the lowest FFT of 0.4949, and the second lowest LPIPS of 0.0972.

While quantitative metrics are a good indicator of performance, they do not fully reflect the actual results. In order to do that, we need to also inspect the produced final images displayed in Figure 20, which shows the mean grown image averaged over 8 rollouts using the final perturbation matrix for each of the losses, along with the target image. All losses, except the SSIM, can successfully recolor the lizard, however, there are visible differences. Huber produces the sharpest looking result consistent with its performance across all metrics, but the limbs and the head have green residue and the edges of the body are much darker compared to others. MSE produces a similar result with slightly better colors. Although it is less sharper and lacks behind in some metrics, the results look to be on par with Huber. The weighted RGBA and the Foreground-masked MSE both produce noticeably blurry results. Comparing the two, weighted RGBA better handles color and does not produce pink eyes. Lastly, although the SSIM has technically the highest SSIM and lowest FFT, it fails the task of turning the lizard blue. This is most likely due to fact that the green lizard to a large degree already satisfies its measures of luminance, contrast, and structure. This is an example of a phenomenon called Goodhart's law [10] which states that when a measure becomes a target, it ceases to be a good measure.

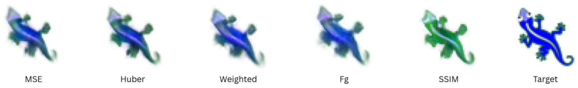

Figure 20: Mean grown image averaged over 8 rollouts using the final perturbation matrix for each of the 5 losses, along with the target image.

Overall, Huber provides the best results closest to the target, both quantitatively and visually. The results show that the selection of loss function affects the quality of the output and the attack in general.

Discussion

Compared to the original backpropagation version, our NCA-ES tuned adversarial attack is less effective in the same time. That said, the evolutionary version is in our opinion surprisingly not that far behind and produces results which are roughly on the same qualitative level as the ones presented by [3].

also put climate results

put qualitative results

Now CMA-ES and the original version are both relatively successfully optimizing the same loss function, and getting to similar results with differing amounts of effort. But there are still differences in the results they produce. The end result of the gradient-optimized attack is an image with more details, but it overemphasizes the blue and is somewhat blurry. The CMA-ES trained adversarial attack leads to a more jagged image that perhaps overemphasizes the green of the edges and is more jagged and grainy, but not washed out.

Crucially, both optimizers optimize in broadly the same direction and our version can further improve on top of the final results by the original approach. Seeing that CMA-ES does not lead away from the optimum but further guides toward it validates our approach. The results would likely be even better if we started the run (see figure 16) with a lower $sigma0$ instead of the priorly found optimal values for full training, so that it hadn't initially taken disproportionately big steps out of the local optimum. It is interesting though that the results of both optimizers do clearly have distinct characteristic looks given the same training objective, meaning they do not optimize in precisely the same direction.

Our approach does not manage to recover from being initialized from a fully gaussian starter matrix, but instead produces a completely white image. These results are very similar to when we first started experimenting with our approach and, underestimating the power of the permutation matrix being multiplied once per growth step for a total of 64-96 times, set $sigma0$ too high. The big implication of small changes likely lock the model in a very unfavorable local optima that it cannot manage to climb out of, as every single step from it makes the results even worse. The standard identity matrix starter is a forgiving start, as it preserves all features of the lizard, with only the colour needing changing, so a step-by-step improvement is possible.

write about all the other results like going from lizard to butterfly

limitations more clearly

future directions & additional possible enhancements

Conclusion

Conclusions. Is your line of work worth pursuing?
we ball

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, vol. 5, no. 2, e23, 2020. [Online]. Available: https://distill.pub/2020/growing-ca doi: 10.23915/distill.00023.

[2] Y. Xu, T. Zhang, and S. Süsstrunk, "AdaNCA: Neural cellular automata as adaptors for more robust vision transformer," *Advances in Neural Information Processing Systems*, vol. 37, pp. 25709–25746, 2024.

[3] E. Randazzo, A. Mordvintsev, E. Niklasson, and M. Levin, "Adversarial reprogramming of neural cellular automata," *Distill*, vol. 6, no. 5, e00027-004, 2021. [Online]. Available: https://distill.pub/selforg/2021/adversarial doi: 10.23915/distill.00027.004.

[4] L. Cavuoti, F. Sacco, E. Randazzo, and M. Levin, "Adversarial takeover of neural cellular automata," in *Artificial Life Conference Proceedings 34*, 2022, p. 38. MIT Press.

[5] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.

[6] S. Greydanus, "Studying growth with neural cellular automata," blog post and PyTorch implementation, 2022. [Online]. Available: https://greydanus.github.io/2022/05/24/studying-growth

[7] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[8] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2018, pp. 586–595.

[9] P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964. [Online]. Available: https://doi.org/10.1214/aoms/1177703732

[10] M. Strathern, "'Improving ratings': audit in the British University system," *European Review*, vol. 5, no. 3, pp. 305–321, 1997. [Online]. Available: https://doi.org/10.1002/(SICI)1234-981X(199707)5:3<305::AID-EURO184>3.0.CO;2-4

Author Contribution

Author contribution: describe each team member contributions to the project.

Code

https://github.com/DavidMagaram/NaturalComputing

Appendix

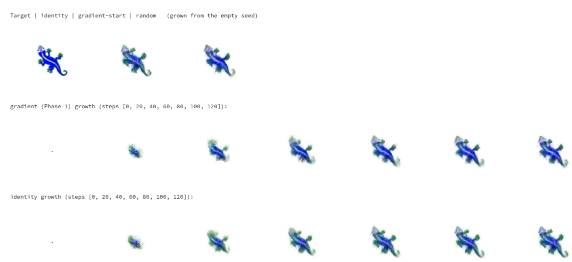

Figure 21: Results from Experiments starting with different matrixes.

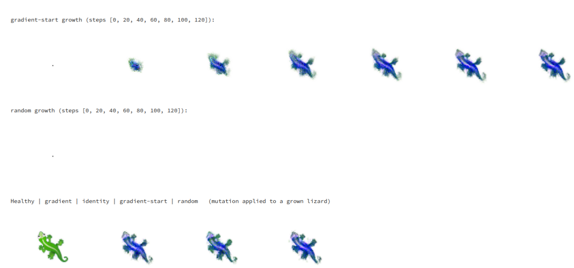

Figure 22

Feature	1 - Poor (Highly Degraded)	2 - Fair (Significant Artifacts)	3 - Good (Minor Flaws)	4 - Excellent (Target Match)
Granularity	Edges are highly unstable; explosive noise, or completely blurred boundaries.	Significant noise or artifacting around the borders; toes are indistinguishable blobs.	Edges are mostly clean; green/dark tint on extremities is visible but somewhat messy or jagged.	Crisp pixelated edges; distinct green fringing correctly placed on the toes/limbs, with minimal to no stray pixels.
Bodily Parts	Unrecognizable mass; missing a tail or multiple limbs.	Recognizable as a lizard, but severely deformed (e.g., missing one leg, severely stunted/bloated tail).	All four limbs and tail are present; minor structural anomalies or slightly incorrect proportions.	Excellent anatomical structure; four distinct legs with toes, a clearly defined head, and a smoothly curving tail.
Blueness	Deep blue is largely absent, completely overtaken by noise, or shifted to the wrong hue.	Blue is present but patchy, washed out, or heavily mixed with incorrect stray colors.	Body is predominantly the correct deep blue; minor color bleeding or slight noise in the torso.	Solid, cohesive deep blue body that contrasts sharply with the dorsal stripe and the head.
Dorsal Stripe	Stripe is completely missing or unrecognizable.	Stripe is visible but heavily fragmented, scattered, or significantly the wrong color/thickness.	Stripe is visible and mostly continuous down the back, but may have minor interruptions or jaggedness.	Distinct, continuous pale white stripe running cleanly from the base of the head down the spine.
Head Features	Head is indistinguishable from the body; eyes are completely missing.	Head is somewhat distinct, but eyes are blurry, merged into a single blob, or missing pupils.	Head shows lighter coloration; two eyes are present but slightly distorted or asymmetric.	Head is distinctively lighter (light purple/blue); two clear white eyes with distinct black pupils are perfectly placed.

Table 5: Visual Fidelity Rubric for the NCA Lizard Target Image.